

Aether — Developer Experience

40+ Commands, Templates, Compose, and More

A CLI built for developers who want power without complexity. Every command is consistent, every output is scriptable, and every workflow is composable.

Built for Developers, by Developers

CLI Overview

Consistent verbs, predictable flags, and multiple output formats for every command.

Core Lifecycle Commands

Command	Description	Example
validate	Check spec against schema	aether validate workload.yaml
build	Build container image	aether build workload.yaml
run	Deploy workload to runtime	aether run workload.yaml
status	Show workload state	aether status my-app
logs	Stream workload logs	aether logs my-app --follow
stop	Stop a running workload	aether stop my-app
destroy	Remove workload and resources	aether destroy my-app
migrate	Move between runtimes	aether migrate my-app kubernetes
scale	Adjust replica count	aether scale my-app --replicas 5
rollback	Revert to previous version	aether rollback my-app

Output Formats & CI-Friendly Flags

Four Output Formats

```
# Default table output
$ aether status

# JSON for scripting
$ aether status -o json | jq '.status'

# YAML for config pipelines
$ aether status -o yaml

# Wide table with extra columns
$ aether status -o wide
```

CI/CD Flags

```
# Dry-run: preview without executing
$ aether run workload.yaml --dry-run
Would deploy my-web-app to podman
  2 replicas, ports: 8080:80
No changes applied.

# Non-interactive: skip all prompts
$ aether destroy my-app --yes
Destroying my-app... done.

# Combine for pipeline validation
$ aether run workload.yaml --dry-run --yes \
  -o json
```

40+

CLI Commands

4

Output Formats

--dry-run

Safe Preview

--yes

CI Mode

Templates & Init

Start new projects instantly with 8 built-in templates or use the interactive wizard.

8 Built-in Templates



web-app

Nginx/Apache with static files, health checks, and TLS.



rest-api

HTTP API server with env config, readiness probes.



database

Postgres/MySQL with persistent volumes and backups.



cache

Redis/Memcached with memory limits and eviction.



worker

Background job processor with queue connection.



cron-job

Scheduled tasks with cron expressions.



ml-training

GPU-enabled training with volume mounts and checkpoints.



microservice

Minimal service with service mesh readiness.

Generate from Template

Quick Scaffold

```
$ aether template web-app --name my-site
```

Generated: my-site/workload.yaml

```
Runtime:    podman
Image:      nginx:alpine
Ports:      8080:80
Health:     HTTP GET /
Resources:  250m CPU, 128Mi RAM
```

Edit the spec, then deploy:

```
$ aether run my-site/workload.yaml
```

Interactive Wizard

```
$ aether init
```

Welcome to Aether!

```
? Workload name: my-api
? Template: rest-api
? Runtime: kubernetes
? Image: node:20-alpine
? Port: 3000
? CPU limit: 500m
? Memory limit: 256Mi
? Replicas: 3
? Enable health check? Yes
```

```
Created: my-api/workload.yaml
Ready to deploy with: aether run
```

Compose & Watch

Deploy entire stacks with dependency ordering, then auto-redeploy on file changes.

Multi-Workload Compose

aether-compose.yaml

```
apiVersion: aether/v1
kind: Compose
metadata:
  name: my-stack
spec:
  workloads:
    database:
      template: database
      image: postgres:16-alpine
      env:
        - name: POSTGRES_DB
          value: appdb
      volumes:
        - pgdata:/var/lib/postgresql/data

    cache:
      template: cache
      image: redis:7-alpine
      depends_on: []

    api:
      template: rest-api
      image: my-registry/api:latest
      ports:
        - containerPort: 3000
          hostPort: 3000
      depends_on:
        - database
        - cache
      env:
        - name: DATABASE_URL
          value: postgres://database:5432/appdb
        - name: REDIS_URL
          value: redis://cache:6379

    frontend:
      template: web-app
      image: my-registry/frontend:latest
      ports:
        - containerPort: 80
          hostPort: 8080
      depends_on:
        - api
```

Deploy the Stack

```
$ aether compose up
```

Resolving dependency order...

1. database (no dependencies)

Watch for Auto-Redeploy

```
$ aether watch --compose aether-compose.yaml
```

Watching for changes...

Patterns: **/*.yaml, **/Dockerfile

2. cache (no dependencies)
3. api (depends: database, cache)
4. frontend (depends: api)

Deploying in order:

- ✓ database Running (1.8s)
- ✓ cache Running (0.9s)
- ✓ api Running (2.1s)
- ✓ frontend Running (1.2s)

Stack my-stack: 4/4 workloads running

Debounce: 2s

[10:05:12] Changed: api/Dockerfile

Rebuilding api...

- ✓ Image built (3.4s)
 - ✓ Rolling restart (2.1s)
- api: Running (v2)

[10:07:45] Changed: workload.yaml

Revalidating spec...

- ✓ Spec valid, applying changes
- ✓ frontend: redeployed (1.1s)

Exec, Port-Forward, Diff & Rollback

Debug, inspect, compare, and revert — all from the same CLI.



aether exec

Execute commands inside a running workload container.

```
$ aether exec my-web-app -- sh

# Now inside the container
/ $ ls /usr/share/nginx/html/
index.html  50x.html  assets/

/ $ cat /etc/os-release
NAME="Alpine Linux"
VERSION_ID=3.19.1

/ $ exit

# One-shot command
$ aether exec my-web-app -- cat /etc/hostname
my-web-app-replica-1
```



aether port-forward

Forward local ports to workload ports for debugging.

```
$ aether port-forward my-web-app 8080:80

Forwarding localhost:8080 → my-web-app:80
Press Ctrl+C to stop

# In another terminal:
$ curl http://localhost:8080
<html>Welcome to nginx!</html>

# Forward multiple ports
$ aether port-forward my-api \
  3000:3000 \
  9229:9229 # Node.js debugger
```



aether diff

Compare the declared spec against the live state. Catch drift before it becomes a problem.

```
$ aether diff my-web-app

Comparing: workload.yaml vs live state

spec.replicas:
- declared: 2
+ live:     3 (manually scaled)

spec.resources.memory:
- declared: 256Mi
+ live:     256Mi (match)

spec.image:
- declared: nginx:alpine
+ live:     nginx:alpine (match)

Drift detected: 1 field(s) differ
Run: aether apply workload.yaml to reconcile
```



aether rollback

Revert to any previous deployment revision instantly.

```
$ aether rollback my-web-app

Available revisions:
#3  current   2026-04-12 10:05  nginx:alpine (3 replicas)
#2          2026-04-12 09:30  nginx:alpine (2 replicas)
#1          2026-04-11 16:00  nginx:1.24

? Rollback to revision: #2

Rolling back my-web-app to revision #2...
✓ Spec restored (2 replicas)
✓ Containers reconciled
✓ Health check passing

Status: Running (revision #2 restored)
```

exec



diagnose



diff



fix



rollback

Every deployment creates a revision snapshot. Rollback restores the full spec, image reference, environment variables, and resource limits — not just the container image.

Supercharge Your Workflow

Shell completions, JSON schema for IDE validation, and a single binary — no runtime dependencies.



Shell Completions

Tab-complete every command, flag, and workload name in your favorite shell.

```
# Bash
$ aether completion bash > \
  /etc/bash_completion.d/aether

# Zsh
$ aether completion zsh > \
  ~/.zfunc/_aether

# Fish
$ aether completion fish > \
  ~/.config/fish/completions/aether.fish

# PowerShell
$ aether completion powershell > \
  $PROFILE.d/aether.ps1

# Reload your shell and enjoy:
$ aether st<TAB>
status stop scale serve secret
```



JSON Schema for IDE Validation

Get real-time validation and autocomplete in VS Code, IntelliJ, and Neovim.

```
# Export the workload schema
$ aether schema workload > \
  aether-workload-schema.json

# VS Code settings.json
{
  "yaml.schemas": {
    "./aether-workload-schema.json": [
      "workload.yaml",
      "aether-compose.yaml"
    ]
  }
}

# Now your editor validates:
# ✓ Required fields
# ✓ Value types and ranges
# ✓ Enum values (runtime, strategy)
# ✓ Autocomplete on properties
```



Single Binary, Zero Dependencies

Aether is a statically-linked Rust binary. No Python, no Node.js, no JVM. Copy it to any Linux machine and it works.

~12MB **~5ms** **~8MB**
Binary Size Startup Time Idle Memory



Full Documentation

- **Getting Started** — install and first deploy
- **CLI Reference** — all 40+ commands detailed
- **YAML Schema** — every field documented
- **Tutorials** — multi-runtime, migration, compose
- **Architecture** — internals and plugin system
- **API Reference** — REST API with OpenAPI spec

One Tool. Every Runtime. Every Workflow.

Aether unifies Podman, Kubernetes, KubeVirt, and Metal3 behind a single CLI with 40+ commands, composable templates, and built-in observability. Install it in seconds and never context-switch between tools again.